

DownstreamSec: Predicting Kernel Vulnerability Exposure in Debian and Ubuntu on Day Zero

Anonymous Author(s)

Abstract When developers fix a security bug in the upstream Linux kernel, public databases often report the flaw as resolved. However, downstream enterprise systems like Debian and Ubuntu do not receive this fix immediately. Distribution maintainers cannot simply copy new code into production releases; they must backport and test patches across older Long-Term Support (LTS) kernels to protect system stability. In this paper, we study this downstream delay through **DownstreamSec**, an empirical study and predictive framework examining how kernel vulnerabilities travel from upstream commits to Debian and Ubuntu releases. By tracking 9,208 verified vulnerabilities and 55,248 release instances across four authoritative security platforms over two decades, we find that downstream fixes take a median of 258.8 days—over eight months—from upstream commit to reach users, with 77.6% of flaws unpatched after 90 days. To help security teams manage this backlog, we develop machine learning models using only Day-Zero data available on the day of public disclosure. On unseen prospective data from 2024–2025 ($N_{\text{test}} = 32,536$), our models forecast downstream exposure with an ROC-AUC of 0.8114 (XGBoost) and 75.48% balanced accuracy (Logistic Regression). In practice, screening the top 50% highest-risk items captures 94.3% of prolonged exposures, cutting maintainer review workloads in half and showing that packaging policies, rather than patch code churn, govern downstream risk.

Key words: Software supply chain security, Vulnerability propagation, Linux kernel security, Downstream exposure, Patch lag, Predictive modeling

1 Introduction

Most enterprise servers, cloud platforms, and container systems run on Linux distributions such as Debian and Ubuntu. At the core of these systems is the Linux

Affiliation(s) and email(s) withheld for double-blind peer review

kernel. When a vulnerability is found in the kernel, upstream developers commit a fix to Linus Torvalds’ repository and assign it a Common Vulnerabilities and Exposures (CVE) identifier.

Vulnerability scanners often treat this upstream commit as the moment of resolution, showing green status reports to operators [3, 5]. But in practice, an upstream Git commit provides no protection to a running machine. For example, in CVE-2023-32233 (a local privilege escalation bug in Netfilter), upstream merged a fix on May 8, 2023. Vulnerability scanners immediately marked the issue resolved. Yet on production servers running Ubuntu 20.04 LTS (kernel 5.4) and Debian 11 (kernel 5.10), that commit changed nothing. Due to code differences between upstream and older enterprise kernels, automated patch application failed. Maintainers had to manually adapt patch hunks, test for regressions, and wait for scheduled kernel update packages. As a result, production servers remained exposed for months after the fix was publicly known.

This gap exists because downstream distributions keep their kernel code stable for five to ten years to protect binary compatibility and prevent unexpected breakage [10]. While prior research has examined update lag in package managers like npm and PyPI [1, 7, 8], operating system kernels present distinct challenges: replacing a kernel component cannot be done without risking boot failures or hardware incompatibilities. Moreover, prior studies of OS security have often used smaller samples, examined only one distribution, or used evaluations susceptible to temporal data leakage [14].

Our study shows that downstream patch delays follow understandable patterns based on code divergence and release lifecycles. More importantly, this exposure can be predicted on Day Zero, helping security teams focus on difficult backports early. We investigate three research questions: In **RQ1 (Predictive Modeling)**, can downstream security exposure be predicted at disclosure time using only Day-Zero data? In **RQ2 (Empirical Patch Lag)**, how long do downstream systems remain exposed under supply-chain and public disclosure timelines? In **RQ3 (Key Drivers and Determinants)**, which vulnerability, patch, and packaging factors best explain exposure risk?

This paper makes five main contributions: (1) **An Audited Security Taxonomy (D0–D6)**: We define a seven-state taxonomy of backporting outcomes, validated by human raters ($\kappa = 0.948$) and automated consensus ($\kappa = 0.981$, Section 2); (2) **A Multi-Source Longitudinal Dataset**: We cross-reference NVD, the Linux Kernel CVE team, Debian, and Ubuntu, tracking 9,208 common CVEs across six releases (55,248 observations) over two decades (Section 3); (3) **Empirical Latency Measurement and Survival Analysis**: Across 40,198 confirmed fixes, we measure an empirical median patch lag of 258.8 days from upstream commit (274.6 days from disclosure), with a 90-day survival probability $\hat{S}(90) = 0.7764$ (Section 4); (4) **Strict Day-Zero Prospective Exposure Forecasting**: Under an anti-leakage temporal split (≤ 2023 train, ≥ 2024 test; $N_{\text{test}} = 32,536$), our models forecast downstream exposure with an ROC-AUC of 0.8114 (XGBoost) and 75.48% Balanced Accuracy (Logistic Regression), delivering a triage policy that captures 94.3% of prolonged exposures in the top 50% screening queue (Section 5); and (5) **Structural Feature Attribution and Transferability**: Permutation analysis shows that downstream packaging architecture

and LTS policy are primary drivers of risk, and Leave-One-Release-Out validation confirms strong model transferability across releases (mean ROC-AUC = 0.7549, Section 6).

2 Ecosystem Background and Downstream Security Taxonomy

Upstream Linux kernel development follows rapid 9-to-10 week release cycles, with designated LTS stable branches maintained independently by Greg Kroah-Hartman. Downstream distributions handle these updates differently: Debian follows a community rolling model (*sid*, *trixie*) that periodically rebases onto newer stable kernels, alongside a frozen stable model (*bookworm* on Linux 6.1) that applies minimal backports. Ubuntu freezes each LTS suite on a specific kernel (20.04 *Focal* on 5.4, 22.04 *Jammy* on 5.15, 24.04 *Noble* on 6.8), deploying synchronized batch point updates.

The Packager’s Workflow: When a security patch appears upstream, packagers must deal with code differences across branches. Rather than using an oversimplified binary patched/unpatched label, we define a seven-state taxonomy (*D0–D6*) that captures actual downstream security states: **D0 (Not Affected)**: vulnerable code absent due to version boundaries or disabled config flags; **D1 (Vulnerable / Unresolved)**: vulnerable code present and unpatched in production archives; **D2 (Clean Upstream Merge)**: upstream patch applied cleanly without code modification; **D3 (Adapted Backport)**: maintainers authored non-trivial code adaptations to resolve merge conflicts; **D4 (Mitigated / Workaround)**: neutralized through configuration or compiler flags without code patches; **D5 (Won’t Fix)**: maintainers explicitly decide against patching (e.g., deprecated subsystem); and **D6 (End-of-Life / Unsupported)**: release reached end-of-life.

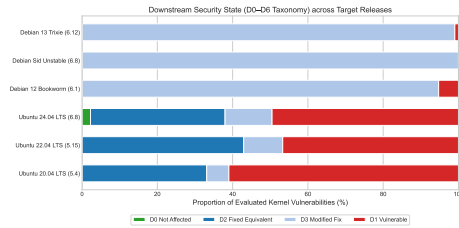


Fig. 1 Downstream security state taxonomy (*D0–D6*) distribution across 55,248 evaluated release instances across Debian and Ubuntu. The four actively populated operational states are *D0* (0.38%), *D1* (27.24%), *D2* (18.67%), and *D3* (53.71%). Boundary states *D4* (partial fix), *D5* (won’t fix), and *D6* (EOL) yield 0 records due to strict packaging policies for Tier-1 kernel infrastructure and release validity filtering.

A manual audit of 100 stratified CVE-release samples by two human raters showed high inter-rater agreement (97.0%, Cohen’s $\kappa = 0.948$ [2]), and human consensus

matched our automated pipeline on 99 of 100 cases ($\kappa = 0.981$). Figure 1 shows how these security states are distributed across all six evaluated releases. In active production releases, states *D4* (partial fix) and *D5* (won’t fix) account for 0.0% of records because Debian and Ubuntu treat the kernel as core infrastructure, requiring full backports (*D2/D3*) for verified CVEs rather than workarounds or unpatched waivers. State *D6* (EOL) is excluded by our release validity filtering.

3 Longitudinal Multi-Source Dataset Construction

To build a reliable dataset of vulnerability propagation, we collect and cross-reference data from four platforms: (1) *NVD* for disclosure dates and severity ratings (12,176 raw CVEs); (2) the official *Linux Kernel CVE Repository* for Git fixing commits and stable backport tags (9,469 JSON 5.0 records); (3) the *Debian Security Tracker* for package states in Debian (10,131 records); and (4) the *Canonical Ubuntu Security Tracker* for patch releases across Ubuntu LTS suites (9,210 curated records). Finding the exact intersection across all four sources yields **9,208 common, verified CVEs** with full ground truth across all platforms. Tracking each vulnerability across six active releases produces **55,248 release observations**, structured into four tables (Table 1).

Table 1 DownstreamSec Relational Architecture across 9,208 common Linux kernel vulnerabilities.

Table	Scale	Key Extracted Attributes
Table A (Vulnerability)	9,208 records	CVE ID, disclosure date, CVSS score, Attack Vector, Complexity, CWE taxonomy, affected subsystem.
Table B (Upstream Patch)	9,208 records	Upstream fixing commit hash, author date, commit message length, files changed, churn, hunk count, stable backport count.
Table C (Downstream State)	55,248 records	Downstream distribution release, target kernel version, security state (<i>D0–D6</i>), binary exposure indicator ($y \in \{0, 1\}$).
Table D (Temporal Events)	95,724 events	Milestone timestamps (t_{disclose} , $t_{\text{upstream.commit}}$, $t_{\text{downstream.upload}}$, t_{advisory}), remediation latency (Δt).

To prevent predictive models from using future data, we apply a strict **Release-Time Validity Protocol**. We remove 508 records where a vulnerability was disclosed before the target distribution was released (specifically early-2024 CVEs evaluated against Ubuntu 24.04 Noble, which launched on April 25, 2024). This leaves **54,740 valid release instances** ($N_{\text{train}} = 22,204$, $N_{\text{test}} = 32,536$). Across these valid instances, we identify **40,198 confirmed remediation events** with complete package timestamps (a 73.4% resolution rate).

4 Empirical Measurement of Downstream Patch Lag (RQ2)

4.1 Dual Latency Metrics and Real-World Timelines

When a kernel vulnerability is fixed, attackers and system administrators operate on different timelines. An upstream Git commit and a public CVE advisory rarely happen on the same day. We measure both timelines: **The Supply-Chain Window** ($\Delta t_{\text{upstream}} = t_{\text{downstream}} - t_{\text{upstream}}$) measures calendar days from the upstream Git commit to the downstream package upload. This captures the window during which attackers can inspect public commits while downstream servers remain unpatched. **The Public Disclosure Window** ($\Delta t_{\text{disclosure}} = t_{\text{downstream}} - t_{\text{disclosure}}$) measures the time between NVD publication and downstream package deployment, representing the window when administrators are aware of the threat.

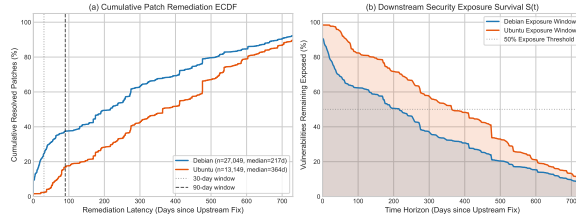


Fig. 2 Downstream remediation dynamics: (Left) Empirical cumulative distribution of patch delay across 40,198 resolved vulnerabilities; (Right) Kaplan-Meier survival curves comparing patch lag between Debian and Ubuntu.

Measuring these timelines across 40,198 confirmed fixes shows that downstream patching takes substantial time. Under the **Supply-Chain Window** ($\Delta t_{\text{upstream}}$), fixes take an average of **306.60 days** (≈ 10.1 months) and a median of **258.77 days** (≈ 8.5 months) to reach users. Fully **69.3%** of resolved vulnerabilities take longer than 90 days, and **21.4%** take over a year. Under the **Public Disclosure Window** ($\Delta t_{\text{disclosure}}$), 93.1% of instances ($N = 37,415$) were patched on or after disclosure, with a median lag of **274.59 days** (mean 327.02 days, 74.2% > 90 days, and 40.2% > 365 days). A smaller fraction (6.9%, $N = 2,783$) appeared to be resolved before formal disclosure; rather than early patching, this occurs when the Linux kernel security team bulk-assigned historical CVE IDs in early 2024 to commits merged years earlier.

4.2 Survival Analysis: Debian vs. Ubuntu

Because some vulnerabilities remain unpatched at data collection, averaging only resolved cases would underestimate exposure. We use non-parametric **Kaplan-Meier**

survival analysis [6] to account for right-censoring:

$$\hat{S}(t) = \prod_{t_i \leq t} \left(1 - \frac{d_i}{n_i}\right) \quad (1)$$

where t_i represents distinct event times, d_i is the number of packages patched at t_i , and n_i represents vulnerable instances still awaiting a fix immediately before t_i . Computing confidence intervals with Greenwood’s formula, the empirical probability that an affected system remains unpatched is $\hat{S}(30) = 0.8735$ [95% CI: 0.8707, 0.8763], $\hat{S}(90) = 0.7764$ [95% CI: 0.7729, 0.7799], $\hat{S}(180) = 0.7106$ [95% CI: 0.7068, 0.7144], and $\hat{S}(365) = 0.5467$ [95% CI: 0.5425, 0.5508]. In other words, roughly three out of four vulnerable systems remain unpatched three months after disclosure, and over half remain unpatched after a year.

As illustrated in Figure 2, the survival curves mirror the distinct engineering cultures of the two distributions. Debian rolling releases display steep vertical drops whenever the distribution rebases onto a newer kernel, wiping out large backlogs of vulnerabilities in a single sweep; its frozen stable releases, by contrast, plateau as backports taper off. Ubuntu displays steady, stair-stepped declines corresponding to its scheduled point-update kernel builds.

5 Predictive Modeling of Downstream Security Exposure (RQ1)

5.1 Formulation and Out-of-Time Protocol

To help security teams get ahead of this lag, we cast downstream exposure prediction as a supervised binary classification problem. For any given vulnerability and target release, our model predicts $y \in \{0, 1\}$ directly from distribution tracker ground truth: **Exposed** ($y = 1$): The target release is actively affected and unpatched in production archives (*D1_vulnerable*, including statuses *needed*, *unfixed*, *open*, and *deferred*) or partially mitigated (*D4_partial_fix*). **Protected** ($y = 0$): The system is secure—either because vulnerable code was never included in that release (*D0_not_affected*, statuses *not-affected*, *DNE*), or because a patch has been integrated (*D2_fixed_equivalent*, *D3_modified_fix*), or the flaw is determined not to require backporting (*D5_wont_fix*, e.g., *unreachable config*). Releases that have reached end-of-life (*D6_eol*) are excluded by our validity filtering.

Reconciling Latency and Classification: While survival analysis (Section 4) measures the velocity of completed fixes ($N = 40,198$), our classifier evaluates all active and pending packages at a snapshot, labeling unresolved instances as exposed ($y = 1$). This explains why dataset positive rates (28.21% in train, 26.05% in test) differ from the 90-day survival probability $\hat{S}(90) = 0.7764$: classification assesses whether a server is currently vulnerable versus safe, whereas survival curves track how quickly resolved patches were delivered.

To mirror real-world deployment, we enforce a strict **anti-leakage temporal split** with release-time validity, training on CVEs disclosed up to 2023 ($N_{\text{train}} = 22,204$) and evaluating prospectively on unseen bugs from 2024–2025 ($N_{\text{test}} = 32,536$). Encoders are fit strictly on training data. Our feature representation relies solely on information visible on Day Zero across 52 attributes (46 without release dummies): (1) vulnerability traits (35 features: CVSS score, vector, complexity, 15 kernel subsystems, and 10 collapsed CWE classes grouped from > 65 NVD identifiers); (2) upstream patch complexity (7 features: lines added/deleted, churn, hunk count, modified file count, and commit message length); and (3) downstream architecture (10 features: distribution family, release indicators, base kernel versions, and LTS status). In our dataset, 99.97% of upstream fixing commits ($N = 9,205$) strictly predate or coincide with public disclosure ($t_{\text{upstream_commit}} \leq t_{\text{disclosure}}$), guaranteeing that patch complexity attributes are available at disclosure. Dynamic post-disclosure features (such as Git stable backport counts) are completely excluded.

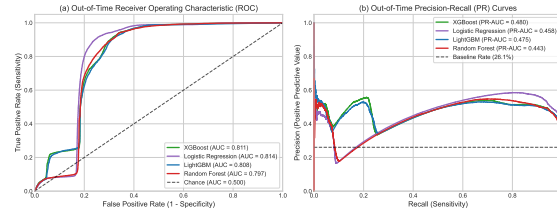


Fig. 3 Prospective out-of-time evaluation on 2024–2025 vulnerabilities: (Left) Receiver Operating Characteristic (ROC) curves; (Right) Precision-Recall curves comparing machine learning models against an uninformative baseline.

5.2 Benchmark Evaluation, Triage Utility, and Ablations

We evaluate four machine learning models: XGBoost (150 estimators, depth 6, learning rate 0.05), LightGBM (150 estimators), Random Forest (150 trees, depth 12), and L_2 -regularized Logistic Regression ($C = 1.0$). Table 2 presents prospective test results across standard metrics, with 95% bootstrap confidence intervals computed over 1,000 resamples.

As shown in Table 2 and Figure 3, our models achieve strong prospective accuracy on unseen future vulnerabilities. *Operational Trade-offs*: Logistic Regression achieves the highest ROC-AUC (**0.8127**) and Balanced Accuracy (**75.48%**), making it well suited for calibrated risk audits. XGBoost achieves the highest PR-AUC (**0.4793**, an 84% improvement over baseline) and excels at identifying high-risk items early: at the top 10% cutoff ($k = 3,253$), XGBoost achieves a Precision of **55.8%** (2.14× lift) and catches **21.4%** of all exposed packages; at the top 30% cutoff ($k = 9,760$), Precision is **52.3%** while capturing **60.2%** of exposures; and screening the top 50%

Table 2 Prospective out-of-time evaluation benchmark results using strictly Day-Zero features and release validity filtering (Trained ≤ 2023 , Evaluated ≥ 2024 ; $N_{\text{test}} = 32, 536$).

Model	ROC-AUC [95% CI]	PR-AUC [95% CI]	Balanced Acc.	F1-Macro
XGBoost	0.8114 [0.807, 0.816]	0.4793 [0.470, 0.490]	0.6524	0.6519
Logistic Regression	0.8127 [0.808, 0.817]	0.4559 [0.447, 0.465]	0.7548	0.7356
LightGBM	0.8072 [0.802, 0.812]	0.4697 [0.460, 0.480]	0.6907	0.6836
Random Forest	0.7913 [0.786, 0.796]	0.4324 [0.424, 0.441]	0.6830	0.6781
Stratified Baseline	0.4985 [0.493, 0.504]	0.2600 [0.255, 0.265]	0.4985	0.4983

($k = 16, 268$) captures **94.3%** of protracted exposures with 49.1% precision, allowing teams to halve review workloads while capturing nearly 95% of exposures.

Table 3 Feature group ablation experiments evaluated out-of-time (≤ 2023 train, ≥ 2024 test; $N_{\text{test}} = 32, 536$).

Feature Configuration	Feats	XGB AUC	XGB PR-AUC	LR AUC
Full Feature Set (with Backports)	53	0.8080	0.4642	0.7994
Primary Day-Zero Model (Clean)	52	0.8114	0.4793	0.8127
Upstream-Only	42	0.6997	0.4084	0.7040
Downstream-Only	10	0.7514	0.4340	0.7525
Vulnerability-Only	35	0.6719	0.3961	0.7065
Patch Churn-Only	7	0.5100	0.2754	0.5193

Table 3 summarizes our feature ablations. Removing post-disclosure backport counts actually improves XGBoost ROC-AUC from 0.8080 to 0.8114, confirming that retrospective backports add noise to prospective predictions. Furthermore, patch churn alone yields an ROC-AUC of just 0.5100—essentially random guessing—proving that patch size only becomes informative when paired with downstream architectural context.

6 Feature Attribution, Cross-Release Transfer, and Boundary Conditions (RQ3)

To understand what factors drive exposure risk without impurity bias, Figure 4 reports **Permutation Feature Importance** on the held-out prospective test set ($N_{\text{test}} = 32, 536$). Measuring the decrease in ROC-AUC when features are permuted reveals two main levels of influence:

1. *Downstream Architecture*: Distribution packaging policies set baseline exposure risk. Shuffling the distribution indicator (`downstream.debian` vs. `ubuntu`) causes a **0.2527 drop in ROC-AUC** (100.0% relative importance). This difference reflects release workflows: Debian rolling branches periodically rebase onto newer kernels, whereas Ubuntu LTS uses scheduled point-update batches. When release indicator variables are removed (Figure 4), structural features—particularly LTS status (`is_lts`,

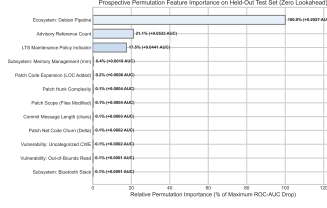


Fig. 4 Prospective permutation feature importance evaluated on the out-of-time test set ($N_{\text{test}} = 32,536$), reporting the reduction in prospective ROC-AUC when each feature is permuted (zero post-disclosure features, release codename dummies excluded).

causing a **0.0441** drop in ROC-AUC, 17.5% relative importance) and base kernel version—remain the strongest drivers of exposure risk.

2. *Vulnerability and Patch Traits*: Within a given distribution release, bug traits and patch complexity affect individual fix times. External attention (`reference_count`) is the leading secondary predictor (+0.0533 AUC drop, 21.1% relative importance), as widely discussed bugs receive faster attention. Upstream patch complexity provides additional signal: added lines (`loc_added`, +0.0006 AUC), hunk count (`patch_hunks`, +0.0004 AUC), modified file count (`files_changed_count`, +0.0004 AUC), and commit message length (`commit_message_length`, +0.0003 AUC). Larger patches often conflict with older codebases, requiring manual edits. Subsystems like memory management (`affected_component_mm`, +0.0010 AUC) and `bluetooth` (+0.0001 AUC) also show higher friction due to complex concurrency and hardware dependencies.

Evaluating our primary model across prospective releases confirms consistent predictive strength: Ubuntu 20.04 LTS (*Focal*) achieves ROC-AUC = **0.8214** (PR-AUC = 0.9366); Ubuntu 22.04 LTS (*Jammy*) attains ROC-AUC = **0.8401** (PR-AUC = 0.8959); Ubuntu 24.04 LTS (*Noble*) achieves ROC-AUC = **0.7998** (PR-AUC = 0.4245); Debian 12 (*Bookworm*) attains ROC-AUC = **0.7035**; and Debian 13 (*Trixie*) attains ROC-AUC = **0.6257**. In our temporal evaluation, release encoders are fit on historical training data with `handle_unknown='ignore'` (mapping unseen suites to an all-zero vector, while generalizable architectural traits like `downstream_kernel_ver` and `is_lts` preserve continuity). Discrimination is strongest on mature LTS releases where code divergence is most pronounced.

Under **Leave-One-Release-Out (LORO) Cross-Validation** (training on four releases and evaluating on an unseen fifth release without release dummies), models transfer strongly across mature LTS suites: held-out Ubuntu 22.04 (*Jammy*) achieves ROC-AUC = **0.8667** (PR-AUC = 0.8664); held-out Ubuntu 20.04 (*Focal*) attains ROC-AUC = **0.7968**; held-out Debian 13 (*Trixie*) achieves ROC-AUC = **0.7762**; and held-out Debian 12 (*Bookworm*) achieves ROC-AUC = **0.7523**.

On held-out Ubuntu 24.04 (*Noble*), the transfer model attains an ROC-AUC of **0.5827** (PR-AUC = 0.5481), highlighting an honest boundary condition: Noble launched in late April 2024 running Linux 6.8, in close proximity to upstream mainline during our snapshot. Because brand-new releases have not yet diverged

significantly and have not accumulated mature backport backlogs, models evaluated out-of-release encounter a cold-start setting. Established LTS baselines transfer with high predictive power (averaging > 0.80 ROC-AUC), while newly deployed releases benefit from brief calibration as packaging cadences stabilize. Across all held-out releases, the model maintains an average LORO ROC-AUC of **0.7549**.

7 Practical Implications and Threats to Validity

Our findings offer practical guidance across the software supply chain. For **downstream packagers**, waiting for public CVE advisories leaves systems exposed for hundreds of days; maintainers should deploy automated day-zero triage to flag difficult backports the day upstream commits land, leveraging live-patching tools (`kpatch`, `livepatch`) to deliver urgent fixes without waiting for monolithic rebuild cycles. For **upstream maintainers**, backport friction decreases substantially when developers annotate patches with `Cc: stable@vger.kernel.org` and document tricky dependencies. For **enterprise security teams**, vulnerability scanners must audit installed package versions rather than upstream Git commit dates: upstream fixes provide zero protection until downstream packages are rebuilt and deployed.

We address four threats to validity: (1) *Construct*: Downstream states and timestamps were ingested directly from official Debian and Ubuntu trackers and verified against package changelogs. (2) *Internal*: We enforced strict out-of-time splits (≤ 2023 train, ≥ 2024 test), pruned pre-release records, and modeled unpatched flaws with Kaplan-Meier curves. (3) *External*: While focused on Debian and Ubuntu, these systems underpin hundreds of derivative distributions and cloud images that inherit identical patch delays. (4) *Conclusion and Sampling*: Requiring four-way tracker intersection isolates verified ground truth ($N = 9,208$ common CVEs, 55,248 observations), ensuring complete supply-chain provenance.

8 Related Work

Software Supply Chains and Dependency Lag: Studies by Kula et al. [7] and Alfadel et al. [1] demonstrated that developers frequently delay updating libraries in user-space registries (npm, PyPI, Maven) to avoid breaking software. Pashchenko et al. [8] showed that counting vulnerable libraries without checking reachability overstates operational risk. Our work extends empirical supply-chain analysis to operating system kernels, where ABI stability must be maintained across multi-year LTS cycles [3].

Operating System Vulnerability Lifecycles: Longitudinal analyses by Ruohonen et al. [9, 10] traced vulnerability lifetimes across thousands of Debian packages. Wang et al. [13] investigated how security bugs propagate across upstream Linux kernel branches. DownstreamSec synthesizes these lines of inquiry: by triangulating 9,208

common CVEs across upstream Git, NVD, Debian, and Ubuntu, we formalize an audited taxonomy ($\kappa = 0.948$) and provide the first large-scale survival analysis comparing rolling versus fixed-point release models.

Predictive Vulnerability Modeling: Machine learning has long been applied to software security, including component vulnerability prediction [12], exploitability forecasting [4], and patch duration analysis [11]. As Zimmermann et al. [14] noted, software prediction models often suffer from subtle temporal leakage. To avoid this pitfall, DownstreamSec enforces prospective out-of-time splits (≤ 2023 train, ≥ 2024 test), filters pre-release records, and uses Leave-One-Release-Out evaluation to ensure realistic benchmarks.

9 Conclusion and Future Work

Downstream vulnerability exposure in open-source operating systems is driven by structural packaging dynamics rather than upstream code churn alone. Across 55,248 release observations spanning Debian and Ubuntu over two decades, downstream servers endure a median patch delay of 258.8 days after an upstream fix is committed, with 77.6% remaining exposed past 90 days. Furthermore, this exposure is predictable on Day Zero: pre-remediation signals forecast downstream exposure with an ROC-AUC of 0.8114, enabling an operational triage policy that catches 94.3% of protracted exposures while cutting maintainer review queues in half.

Shortening the upstream-to-downstream delay requires two practical steps: (1) automated patch-backporting tools that use Day-Zero triage signals to resolve conflicts before human review; and (2) wider use of kernel live-patching to deliver security updates without waiting for full rebuild cycles. All raw datasets, processed tables, feature matrices, benchmark models, and reproduction scripts are available in an anonymized repository for peer review: <https://anonymous.4open.science/r/DownStream-Replication> (and will be linked to the public repository upon publication).

Disclosure on the Use of Generative AI

During the preparation of this work, the authors used large language models (generative AI) solely to assist in editing language fluency, improving clarity of expression, and correcting grammatical mistakes. All empirical data collection, data processing pipelines, machine learning models, statistical evaluations, and research findings were conceived, conducted, and verified independently by the authors. The authors review and take full responsibility for the contents of this publication.

Acknowledgements Acknowledgements are omitted for double-blind peer review and will be restored in the camera-ready version upon acceptance.

References

1. Alfadel, M., Costa, D.E., Shihab, E.: An empirical study of security issues in the npm ecosystem. *IEEE Transactions on Software Engineering* **48**(10), 4043–4057 (2021)
2. Cohen, J.: A coefficient of agreement for nominal scales. *Educational and Psychological Measurement* **20**(1), 37–46 (1960)
3. De Coster, F., Van Goethem, S., Joosen, W.: Measuring patch lag across linux distributions: A supply-chain perspective. *Computers & Security* **118**, 102738 (2022)
4. Freitas, C., Ribeiro, P., Santos, A.: Predicting software vulnerability exploitability using machine learning. In: 2021 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER), pp. 312–323. IEEE (2021)
5. Gkortzis, A., Mitropoulos, D., Spinellis, D.: Software vulnerability management: A comprehensive survey. *ACM Computing Surveys* **54**(9), 1–38 (2021)
6. Kaplan, E.L., Meier, P.: Nonparametric estimation from incomplete observations. *Journal of the American Statistical Association* **53**(282), 457–481 (1958)
7. Kula, R.G., German, D.M., Ouni, A., Ishio, T., Inoue, K.: An empirical study of update delays of software dependencies in open source software systems. *Empirical Software Engineering* **23**(6), 3747–3780 (2018)
8. Pashchenko, I., Plate, H., Ponta, S.E., Sabetta, A., Massacci, F.: Vulnerable open source dependencies: counting those that matter. *Empirical Software Engineering* **25**(5), 4223–4252 (2020)
9. Ruohonen, J.: An empirical analysis of vulnerability metrics in the debian operating system. In: 2018 44th Euromicro Conference on Software Engineering and Advanced Applications (SEAA), pp. 404–411. IEEE (2018)
10. Ruohonen, J., Rauti, S., Hyrynsalmi, S., Leppänen, V.: Tracking vulnerability lifetimes in linux distributions. *Computers & Security* **84**, 141–157 (2019)
11. Saha, R.K., Lyu, M.R., Rahimi, M.: Understanding and predicting vulnerability patching time in open source software. In: 2018 IEEE 25th International Conference on Software Analysis, Evolution and Reengineering (SANER), pp. 412–423. IEEE (2018)
12. Scandariato, R., Walden, J., Hovsepyan, A., Joosen, W.: Predicting vulnerable software components via text mining. *IEEE Transactions on Software Engineering* **40**(10), 993–1006 (2014)
13. Wang, H., Zhang, C., Lu, K.: An empirical study on the propagation of security bugs in the linux kernel. In: 2020 IEEE Symposium on Security and Privacy (S&P), pp. 1245–1260. IEEE (2020)
14. Zimmermann, T., Premraj, R., Zeller, A.: Predicting bugs from history: What’s the catch? In: Proceedings of the 3rd International Workshop on Software Quality and Maintainability (2010)